

result that indicates death after birth. You can add such a constraint for each Author model through the form classes. You can even define validators for each model field. All the validators are used at the form level by the model's `clean()` method.

Now, let's discuss how to structure your tests.

Testing Structure

Django leverages unit test module for identifying the inbuilt test codes. There is a self-discovery module in Django for the unit test, which has a working directory named `test*.py`, where `py` stands for Python. The reason behind choosing a Python standard is its capability of automating the tests. The best way to use pre-built test codes in Django is to create modules accordingly. You can also do other modules for testing custom test codes. Please note that keeping separate files for models, views, and forms can reduce your testing effort.

- `catalog/`
- `/tests/`
- `__init__.py`
- `test_models.py`
- `test_forms.py`
- `Test_views.py`

Developers can create such a file structure in their project directory. It is essential to indicate that the `__init__.py` is an empty file, as it allows the Python to project that a directory is a package. Once you create a web app, Django creates a skeleton test code files- `/catalog/tests.py`.

You can just copy the skeleton files and rename them in your directory according to the project. The next thing you will like to do is delete the skeleton files to make your test file size bulky.

Next, the files in the directory will import the Django test code:

- `from Django.test import TestCase`
- `# Create your tests here.`
- `There are two ways to go about structuring the tests:`
- `The first is to add a test class for all the models or`